NLP Assignment 2

Gaurav Rajpurohit
Roll No: MT2025047

Problem 1a

In this question, a transition-based dependency parser was implemented to parse the sentence:

"I parsed this sentence correctly"

Initially, the stack contains only `ROOT`, the buffer contains all the words of the sentence, and the dependency list is empty. The parser then applies three transitions:

- **SHIFT** – moves the first word from the buffer to the stack.
- **LEFT-ARC** – makes the top word the head of the second word in the stack.
- **RIGHT-ARC** – makes the second word the head of the top word in the stack.

Using these transitions, the parser gradually builds the dependency tree. The sequence of configurations for the stack and buffer is shown below.

Stack	**Buffer**	**Transition**
[ROOT]	[I, parsed, this, sentence, correctly]	Initial
[ROOT, I]	[parsed, this, sentence, correctly]	SHIFT
[ROOT, I, parsed]	[this, sentence, correctly]	SHIFT
[ROOT, parsed]	[this, sentence, correctly]	LEFT-ARC
[ROOT, parsed, this]	[sentence, correctly]	SHIFT
[ROOT, parsed, this, sentence]	[correctly]	SHIFT
[ROOT, parsed, sentence]	[correctly]	LEFT-ARC
[ROOT, parsed]	[correctly]	RIGHT-ARC
[ROOT, parsed, correctly]	[ ]	SHIFT
[ROOT, parsed]	[ ]	RIGHT-ARC
[ROOT]	[ ]	RIGHT-ARC

Table 1: Transition steps during dependency parsing

After applying all transitions, the final dependencies obtained are:

- parsed $\rightarrow$ I
- sentence $\rightarrow$ this
- parsed $\rightarrow$ sentence
- parsed $\rightarrow$ correctly
- ROOT $\rightarrow$ parsed

The implementation of the parser was done in Python using a stack, buffer, and dependency list. The following screenshots show different parts of the implementation and the output generated by the parser.

```python
import spacy

# Load spaCy English model
nlp = spacy.load("en_core_web_sm")


def get_spacy_dependencies(sentence):
    """
    Uses spaCy to extract head-dependent relations
    which will act as the 'gold tree'.
    """
    doc = nlp(sentence)
    gold_deps = []

    for token in doc:
        if token.dep_ == "ROOT":
            gold_deps.append(("ROOT", token.text))
        else:
            gold_deps.append((token.head.text, token.text))

    return gold_deps


class DependencyParser:
    def __init__(self, sentence):
        self.stack = ['ROOT']
        self.buffer = sentence.split()
        self.dependencies = []

    def shift(self):
        if self.buffer:
            self.stack.append(self.buffer.pop(0))

    def left_arc(self):
        if len(self.stack) >= 2:
            head = self.stack[-1]
            dependent = self.stack[-2]
            self.dependencies.append((head, dependent))
            self.stack.pop(-2)

    def right_arc(self):
        if len(self.stack) >= 2:
            head = self.stack[-2]
            dependent = self.stack[-1]
            self.dependencies.append((head, dependent))
            self.stack.pop(-1)
```

```python
def oracle(sentence, gold_dependencies):

    buffer = sentence.split()
    stack = ['ROOT']
    transitions = []

    while len(buffer) > 0 or len(stack) > 1:

        action_taken = False

        if len(stack) >= 2:

            s1 = stack[-1]
            s2 = stack[-2]

            # LEFT ARC
            if (s1, s2) in gold_dependencies:
                transitions.append("LEFT-ARC")
                stack.pop(-2)
                action_taken = True

            # RIGHT ARC
            elif (s2, s1) in gold_dependencies:

                has_dependents_in_buffer = any(
                    (s1, word) in gold_dependencies for word in buffer
                )

                if not has_dependents_in_buffer:
                    transitions.append("RIGHT-ARC")
                    stack.pop(-1)
                    action_taken = True

        # SHIFT
        if not action_taken:
            if len(buffer) > 0:
                transitions.append("SHIFT")
                stack.append(buffer.pop(0))
            else:
                break

    return transitions
```

```python
sentence = "I parsed this sentence correctly"

print("Sentence:", sentence)

# Get dependencies automatically
gold_deps = get_spacy_dependencies(sentence)

print("\nspaCy Extracted Dependencies:")
for dep in gold_deps:
    print(f"{dep[0]} -> {dep[1]}")

# Generate transitions automatically
generated_transitions = oracle(sentence, gold_deps)

print("\nTransitions Generated by Oracle:")
print(generated_transitions)

# Run parser
parser = DependencyParser(sentence)

print("\nParser Execution:")
print(f"{'Stack':<35} | {'Buffer':<40} | {'Transition'}")
print("-" * 90)

for step in generated_transitions:

    if step == "SHIFT":
        parser.shift()

    elif step == "LEFT-ARC":
        parser.left_arc()

    elif step == "RIGHT-ARC":
        parser.right_arc()

    print(f"{str(parser.stack):<35} | {str(parser.buffer):<40} | {step}")

print("\nFinal Dependencies Built by Parser:")
for dep in parser.dependencies:
    print(dep)
```

```
Sentence: I parsed this sentence correctly

spaCy Extracted Dependencies:
parsed -> I
ROOT -> parsed
sentence -> this
parsed -> sentence
parsed -> correctly

Transitions Generated by Oracle:
['SHIFT', 'SHIFT', 'LEFT-ARC', 'SHIFT', 'SHIFT', 'LEFT-ARC', 'RIGHT-ARC', 'SHIFT', 'RIGHT-ARC', 'RIGHT-ARC']

Parser Execution:
Stack                               | Buffer                                   | Transition
------------------------------------------------------------------------------------------
['ROOT', 'I']                       | ['parsed', 'this', 'sentence', 'correctly'] | SHIFT
['ROOT', 'I', 'parsed']             | ['this', 'sentence', 'correctly']        | SHIFT
['ROOT', 'parsed']                  | ['this', 'sentence', 'correctly']        | LEFT-ARC
['ROOT', 'parsed', 'this']          | ['sentence', 'correctly']                | SHIFT
['ROOT', 'parsed', 'this', 'sentence'] | ['correctly']                         | SHIFT
['ROOT', 'parsed', 'sentence']      | ['correctly']                            | LEFT-ARC
['ROOT', 'parsed']                  | ['correctly']                            | RIGHT-ARC
['ROOT', 'parsed', 'correctly']     | []                                       | SHIFT
['ROOT', 'parsed']                  | []                                       | RIGHT-ARC
['ROOT']                            | []                                       | RIGHT-ARC

Final Dependencies Built by Parser:
('parsed', 'I')
('sentence', 'this')
('parsed', 'sentence')
('parsed', 'correctly')
('ROOT', 'parsed')
```

```python
from spacy import displacy

def visualize_dependency_tree(sentence):

    doc = nlp(sentence)

    print("\nDependency Tree Visualization:")

    displacy.render(
        doc,
        style="dep",
        jupyter=True,
        options={
            "distance":120,
            "compact":False,
            "bg":"#ffffff",
            "color":"#000000"
        }
    )

# Run visualization
visualize_dependency_tree(sentence)
```

```
Dependency Tree Visualization:
```

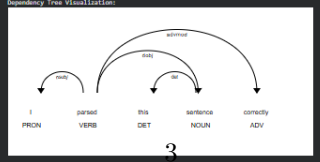

Figure 1: Implementation and output of the transition-based dependency parser

Problem 1b

For a sentence containing n words, the transition-based dependency parser requires a total of $2n$ **steps** to complete the parsing process.

Initially, all the words of the sentence are placed in the buffer while the stack contains only `ROOT`. During parsing, the parser applies two main types of operations:

- **SHIFT**: moves a word from the buffer to the stack.
- **ARC operations** (**LEFT-ARC** or **RIGHT-ARC**): create dependency relations between words and remove elements from the stack.

For a sentence with n words:

- Each word must be moved from the buffer to the stack exactly once. Therefore, n **SHIFT operations** are required.
- Each word must form one dependency relation with its head in the dependency tree (including the ROOT dependency). Therefore, n **ARC operations** are required.

Hence, the total number of transitions needed is:

$$\text{Total steps} = n\ (\text{SHIFT}) + n\ (\text{ARC}) = 2n$$

Therefore, a transition-based dependency parser requires $2n$ **transitions** to parse a sentence containing n words.

Exercise 2: LSTM and GRU

Problem 2a: Adjusting LSTM Hyperparameters

To improve the performance of an LSTM network, several hyperparameters can be tuned depending on the task. Important hyperparameters include input size, hidden size, number of layers, output size, and dropout rate. These parameters help control the capacity of the model and its ability to learn patterns from sequential data.

In this implementation, an optimized LSTM model was created using PyTorch. The model includes an LSTM layer followed by a fully connected layer. Dropout is also applied between LSTM layers to reduce overfitting.

Hyperparameters Used

- **Input Size**: Number of input features given to the model at each time step.
- **Hidden Size**: Number of neurons in the hidden layer that capture sequence patterns.

- **Number of Layers**: Stacking multiple LSTM layers helps learn deeper sequential representations.
- **Output Size**: Number of values predicted by the model.
- **Dropout Rate**: Used to prevent overfitting by randomly dropping some neurons during training.

Two different tasks were considered to demonstrate how hyperparameters can be adjusted.

1. Time Series Prediction

For time series prediction tasks such as stock price or weather forecasting:

- Input Size = 1 (single feature)
- Hidden Size = 64
- Number of Layers = 2
- Output Size = 1
- Dropout = 0.2

These values are sufficient for learning temporal patterns in numerical sequences.

2. Text Generation

For text generation tasks such as predicting the next word in a sentence:

- Input Size = 256 (embedding dimension)
- Hidden Size = 512
- Number of Layers = 3
- Output Size = Vocabulary Size
- Dropout = 0.5

Larger hidden layers and more layers help the model capture complex language patterns.

Problem 2b: How LSTM Solves the Vanishing Gradient Problem

Traditional Recurrent Neural Networks (RNNs) suffer from the vanishing gradient problem when learning long sequences. During backpropagation through time, the gradients may become very small, making it difficult for the network to learn long-term dependencies between words.

LSTM networks solve this problem by introducing a memory cell and three gating mechanisms that control the flow of information through the network.

Forget Gate

The forget gate decides which information from the previous cell state should be removed. It takes the previous hidden state and the current input and passes them through a sigmoid activation function. The output determines how much of the previous information should be kept or forgotten.

Input Gate

The input gate determines which new information should be added to the memory cell. It selects important features from the current input and updates the cell state accordingly. This helps the network store useful information for future steps.

Output Gate

The output gate controls what information from the cell state should be passed to the next hidden state. It decides which part of the stored information will be used as the output for the current time step.

Example: Sequence Modelling

Consider the sentence:

"The movie was very long but the ending was excellent."

Here, the word *excellent* plays an important role in determining the sentiment. LSTM can remember such important information through its memory cell and gates, allowing it to correctly capture long-term dependencies in the sequence.

Problem 2c: GRU Network with Reset and Update Gates

The GRU architecture uses two gates:

- Reset Gate
- Update Gate

$$z_t = \sigma(W_z x_t + U_z h_{t-1}) \tag{1}$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1}) \tag{2}$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1})) \tag{3}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{4}$$

```python
import torch
import torch.nn as nn

class CustomGRU(nn.Module):
    def __init__(self, input_size, hidden_size):
        super(CustomGRU, self).__init__()

        self.hidden_size = hidden_size

        # Linear layer to compute Update gate (z_t) and Reset gate (r_t)
        # [x_t , h_(t-1)] → 2 * hidden_size
        self.gate_linear = nn.Linear(input_size + hidden_size, 2 * hidden_size)

        # Linear layer for candidate hidden state
        self.candidate_linear = nn.Linear(input_size + hidden_size, hidden_size)

    def forward(self, x, h_prev):

        # Combine input and previous hidden state
        combined = torch.cat((x, h_prev), dim=1)

        # ------------------------------------------------
        # Update Gate and Reset Gate
        # z_t = σ(Wz[x_t , h_(t-1)])
        # r_t = σ(Wr[x_t , h_(t-1)])
        # ------------------------------------------------
        gates = torch.sigmoid(self.gate_linear(combined))
        z_t, r_t = gates.chunk(2, dim=1)

        # ------------------------------------------------
        # Candidate hidden state
        # h~_t = tanh(Wh[x_t , r_t * h_(t-1)])
        # ------------------------------------------------
        combined_reset = torch.cat((x, r_t * h_prev), dim=1)
        h_candidate = torch.tanh(self.candidate_linear(combined_reset))

        # ------------------------------------------------
        # Final hidden state
        # h_t = (1 - z_t) * h_(t-1) + z_t * h~_t
        # ------------------------------------------------
        h_next = (1 - z_t) * h_prev + z_t * h_candidate

        return h_next


# Example parameters
input_size = 10
hidden_size = 50

# Create GRU cell
gru_cell = CustomGRU(input_size, hidden_size)

# Dummy input to test
sample_input = torch.randn(1, input_size)
sample_hidden = torch.randn(1, hidden_size)

output = gru_cell(sample_input, sample_hidden)

print("Output hidden state shape:", output.shape)
```

```
Output hidden state shape: torch.Size([1, 50])
```

Figure 2: GRU Implementation with Reset and Update Gates

Problem 2d: Advantages and Limitations of GRU for Sentiment Analysis

Gated Recurrent Unit (GRU) networks are commonly used for natural language processing tasks such as sentiment analysis of social media posts, product reviews, and tweets. Since text data is sequential in nature, GRU models are able to capture the contextual relationships between words in a sentence and use this

information to determine the overall sentiment.

Advantages

- GRU has a simpler architecture compared to LSTM because it uses only two gates (reset gate and update gate). This makes the model easier to train.
- It requires fewer parameters than LSTM, which reduces computational cost and training time.
- GRU can effectively capture sequential dependencies in text data, which is important for identifying sentiment in sentences.
- It performs particularly well on short to medium length sentences, which are commonly found in social media posts and online reviews.

Limitations

- GRU may not perform as well as LSTM for very long sequences where long-term memory is required.
- Because it has fewer gates, GRU provides less control over memory compared to LSTM.
- In cases where complex hierarchical relationships exist in text, more advanced architectures such as LSTM or transformer-based models may perform better.

Overall, GRU offers a good balance between computational efficiency and performance. It is widely used for sentiment analysis tasks where fast training and reasonable accuracy are required, especially when working with large amounts of social media text data.

0.1 GITHUB LINK

https://github.com/Gaurav-Rajpurohit/NLP-Assignment2/blob/main/Assignment.ipynb